

IFRN — Instituto Federal do Rio Grande do Norte
Introdução ao Django Rest Framework

Novos Modelos, Ferramentas de Teste e Documentação

TADS — 4º Semestre | Prof. Plácido Neto

Objetivos desta Extensão

Ao final desta aula complementar o aluno será capaz de:

- Criar dois novos modelos Django independentes (sem relacionamento entre si): Vendedor e Produto;
- Implementar Serializers, ViewSets e rotas para os novos modelos;
- Testar os endpoints da API com três ferramentas distintas:
 - Swagger UI (interface web gerada automaticamente pelo drf-yasg);
 - REST Client (extensão do Visual Studio Code);
 - Insomnia / Postman (clientes REST dedicados).

11. Novos Modelos: Vendedor e Produto

Na aula anterior criamos o modelo Cliente. Agora adicionaremos mais dois modelos independentes ao aplicativo backend: Vendedor (representando quem vende itens na plataforma) e Produto (o catálogo de itens disponíveis). Esses modelos não possuem relacionamentos entre si nem com Cliente — serão conectados por **ForeignKey** em aulas futuras.

11.1 Modelo Vendedor

O modelo Vendedor armazena dados cadastrais e comerciais de quem oferta produtos na plataforma.

```
# backend/models.py (adicionar após a classe Cliente)

class Vendedor(models.Model):

    nome = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    cpf_cnpj = models.CharField(max_length=18, unique=True)
    telefone = models.CharField(max_length=20, blank=True)
    avaliacao = models.DecimalField(
        max_digits=3,
        decimal_places=2,
```

```

        default=5.00    # Nota inicial máxima
    )
    ativo               = models.BooleanField(default=True)
    data_cadastro       = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = 'vendedores'
        ordering = ['nome']

    def __str__(self):
        return f'{self.nome} ({self.cpf_cnpj})'

```

Código 11.1 — Modelo Vendedor em backend/models.py

Campo	Tipo Django	SQL Gerado	Observação
nome	CharField(100)	VARCHAR(100)	Nome completo
email	EmailField(unique)	VARCHAR(254) UNIQUE	Formato validado
cpf_cnpj	CharField(18, unique)	VARCHAR(18) UNIQUE	CPF ou CNPJ
telefone	CharField(20, blank)	VARCHAR(20)	Campo opcional
avaliacao	DecimalField(3,2)	NUMERIC(3,2)	De 0.00 a 5.00
ativo	BooleanField(default=True)	BOOLEAN	
data_cadastro	DateTimeField(auto_now_add)	TIMESTAMP	Automático

Tabela 11.1 — Campos do modelo Vendedor

11.2 Modelo Produto

O modelo Produto representa o catálogo de itens disponíveis na plataforma, com informações de preço, estoque e categoria.

```

# backend/models.py (adicionar após a classe Vendedor)

class Produto(models.Model):

    CATEGORIA_CHOICES = [
        ('eletronicos', 'Eletrônicos'),
        ('roupas',      'Roupas e Acessórios'),
        ('livros',       'Livros'),
        ('alimentos',    'Alimentos'),
        ('outros',       'Outros'),
    ]

    nome           = models.CharField(max_length=200)
    descricao      = models.TextField(blank=True)
    preco          = models.DecimalField(max_digits=10, decimal_places=2)
    estoque        = models.IntegerField(default=0)

```

```

    categoria = models.CharField(
        max_length=20,
        choices=CATEGORIA_CHOICES,
        default='outros'
    )

    disponivel = models.BooleanField(default=True)
    criado_em = models.DateTimeField(auto_now_add=True)
    atualizado_em = models.DateTimeField(auto_now=True) # Atualizado a cada save()

    class Meta:
        db_table = 'produtos'
        ordering = ['nome']

    def __str__(self):
        return f'[{self.nome} - R$ {self.preco}]'

```

Código 11.2 — Modelo Produto em backend/models.py

Campo	Tipo Django	SQL Gerado	Observação
nome	CharField(200)	VARCHAR(200)	Nome do produto
descricao	TextField(blank)	TEXT	Descrição longa
preco	DecimalField(10,2)	NUMERIC(10,2)	Ex.: 1299.90
estoque	IntegerField(default=0)	INTEGER	Qtde disponível
categoria	CharField + choices	VARCHAR(20)	Valores predefinidos
disponivel	BooleanField(default=True)	BOOLEAN	Ativo/Inativo
criado_em	DateTimeField(auto_now_add)	TIMESTAMP	Criação automática
atualizado_em	DateTimeField(auto_now)	TIMESTAMP	Atualização automática

Tabela 11.2 — Campos do modelo Produto

 Diferença entre `auto_now_add` e `auto_now`: `auto_now_add` preenche o campo apenas na criação do registro (INSERT); `auto_now` atualiza o campo em todo `save()` (UPDATE). Por isso `atualizado_em` sempre reflete a última modificação.

12. Serializers para Vendedor e Produto

Adicione os dois novos serializers ao arquivo `backend/serializers.py`, seguindo o mesmo padrão do `ClienteSerializer`.

```

# backend/serializers.py

from rest_framework import serializers
from .models import Cliente, Vendedor, Produto

```

```

class ClienteSerializer(serializers.ModelSerializer):
    class Meta:
        model = Cliente
        fields = '__all__'

class VendedorSerializer(serializers.ModelSerializer):
    class Meta:
        model = Vendedor
        fields = '__all__'

class ProdutoSerializer(serializers.ModelSerializer):
    categoria_display = serializers.CharField(
        source='get_categoria_display',
        read_only=True
    )

    class Meta:
        model = Produto
        fields = '__all__' # inclui categoria_display automaticamente
        extra_fields = ['categoria_display']

```

Código 12.1 — Serializers completos em backend/serializers.py

💡 O campo `categoria_display` usa `source='get_categoria_display'` — um método gerado automaticamente pelo Django para campos com choices. Ele retorna o label legível ('Eletrônicos') em vez do valor interno ('eletronicos').

13. ViewSets e URLs

Adicione os dois ViewSets ao arquivo `backend/views.py` e registre as novas rotas no roteador global.

13.1 backend/views.py

```

# backend/views.py

from rest_framework import viewsets
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import SearchFilter, OrderingFilter

from .models import Cliente, Vendedor, Produto
from .serializers import ClienteSerializer, VendedorSerializer, ProdutoSerializer

class ClienteViewSet(viewsets.ModelViewSet): # (mantido da aula anterior)
    queryset = Cliente.objects.all()

```

```

        serializer_class = ClienteSerializer

class VendedorViewSet(viewsets.ModelViewSet):
    queryset = Vendedor.objects.all()
    serializer_class = VendedorSerializer

class ProdutoViewSet(viewsets.ModelViewSet):
    queryset = Produto.objects.filter(disponivel=True) # apenas ativos
    serializer_class = ProdutoSerializer

```

Código 13.1 — ViewSets em backend/views.py

13.2 amazon/urls.py — Registrando as Novas Rotas

```

# amazon/urls.py

from django.contrib import admin
from django.urls import path, include
from rest_framework.routers import DefaultRouter
from backend import views

# Swagger imports (da aula anterior)
from drf_yasg.views import get_schema_view
from drf_yasg import openapi
from rest_framework import permissions

router = DefaultRouter()
router.register(r'clientes', views.ClienteViewSet, basename='cliente')
router.register(r'vendedores', views.VendedorViewSet, basename='vendedor') # NOVO
router.register(r'produtos', views.ProdutoViewSet, basename='produto') # NOVO

schema_view = get_schema_view(
    openapi.Info(
        title='Amazon API',
        default_version='v1',
        description='API RESTful para gerenciamento de pedidos',
    ),
    public=True,
    permission_classes=(permissions.AllowAny,),
)

urlpatterns = [
    path('admin/', admin.site.urls),
    path('amazon_api/', include(router.urls)),
    path('swagger/', schema_view.with_ui('swagger', cache_timeout=0),
name='swagger-ui'),
    path('redoc/', schema_view.with_ui('redoc', cache_timeout=0),
name='redoc-ui'),
]

```

Código 13.2 — Roteamento global com os três recursos em amazon/urls.py

13.3 Aplicando as Migrações

```
# Gerar os arquivos de migração dos novos modelos
python manage.py makemigrations
```

```
# Saída esperada:
# Migrations for 'backend':
#   backend/migrations/0002_vendedor_produto.py
#       + Create model Vendedor
#       + Create model Produto
```

```
# Aplicar ao banco de dados
python manage.py migrate
```

```
# Iniciar o servidor
python manage.py runserver
```

Terminal 13.1 — Comandos para aplicar as migrações dos novos modelos

Ação	Método	URL	Descrição
list	GET	/amazon_api/vendedores/	Lista todos os vendedores
create	POST	/amazon_api/vendedores/	Cadastra novo vendedor
retrieve	GET	/amazon_api/vendedores/{id}/	Detalha um vendedor
update	PUT	/amazon_api/vendedores/{id}/	Atualiza todos os campos
partial	PATCH	/amazon_api/vendedores/{id}/	Atualiza campos parciais
destroy	DELETE	/amazon_api/vendedores/{id}/	Remove o vendedor
list	GET	/amazon_api/produtos/	Lista todos os produtos
create	POST	/amazon_api/produtos/	Cadastra novo produto
retrieve	GET	/amazon_api/produtos/{id}/	Detalha um produto
update	PUT	/amazon_api/produtos/{id}/	Atualiza todos os campos
partial	PATCH	/amazon_api/produtos/{id}/	Atualiza campos parciais
destroy	DELETE	/amazon_api/produtos/{id}/	Remove o produto

Tabela 13.1 — Endpoints gerados automaticamente pelo DefaultRouter para Vendedor e Produto

14. Testando com o Swagger UI

O Swagger UI, configurado com o pacote drf-yasg na aula anterior, exibe automaticamente os novos endpoints após o registro dos ViewSets. Nenhuma configuração adicional é necessária.

14.1 Acessando a Documentação

Com o servidor em execução, abra no navegador:

```
http://localhost:8000/swagger/
```

A interface exibirá três seções expansíveis: clientes, vendedores e produtos. Cada uma lista todas as operações disponíveis com seus schemas de entrada e saída.

14.2 Testando POST — Criar um Vendedor

Passo a passo para cadastrar um vendedor diretamente pelo Swagger:

- Clique na seção vendedores para expandi-la;
- Clique em POST /amazon_api/vendedores/;
- Clique no botão Try it out (canto direito);
- Substitua o corpo de exemplo pelo JSON abaixo e clique em Execute:

```
{
  "nome":      "Carlos Menezes",
  "email":     "carlos@loja.com",
  "cpf_cnpj":  "123.456.789-00",
  "telefone":  "(84) 98765-4321",
  "avaliacao": "4.80",
  "ativo":     true
}
```

JSON 14.1 — Corpo da requisição para criar um Vendedor

Resposta esperada (HTTP 201 Created):

```
{
  "id":        1,
  "nome":      "Carlos Menezes",
  "email":     "carlos@loja.com",
  "cpf_cnpj":  "123.456.789-00",
  "telefone":  "(84) 98765-4321",
  "avaliacao": "4.80",
  "ativo":     true,
  "data_cadastro": "2025-06-01T14:30:00Z"
}
```

JSON 14.2 — Resposta HTTP 201 do Swagger após criação do Vendedor

14.3 Testando POST — Criar um Produto

```
{
  "nome":      "Teclado Mecânico RGB",
  "descricao": "Teclado mecânico com switches blue e iluminação RGB.",
  "preco":     "349.90",
  "estoque":   50,
  "categoria": "eletronicos",
  "disponivel": true
}
```

JSON 14.3 — Corpo da requisição para criar um Produto

Resposta esperada (HTTP 201 Created):

```
{
  "id":      1,
  "nome":    "Teclado Mecânico RGB",
  "descricao": "Teclado mecânico com switches blue e iluminação RGB.",
  "preco":   "349.90",
  "estoque": 50,
  "categoria": "eletronicos",
  "categoria_display": "Eletrônicos",
  "disponivel": true,
  "criado_em": "2025-06-01T14:35:00Z",
  "atualizado_em": "2025-06-01T14:35:00Z"
}
```

JSON 14.4 — Resposta HTTP 201 com campo categoria_display preenchido

14.4 Testando GET com Filtros

O Swagger também permite testar os parâmetros de query string gerados pelo django-filter. Clique em GET /amazon_api/produtos/ → Try it out e preencha os campos de filtro:

Parâmetro	Valor Exemplo	Efeito
categoria	eletronicos	Filtra apenas produtos da categoria
disponivel	true	Filtra apenas produtos disponíveis
search	Teclado	Busca textual em nome, descricao
ordering	preco	Ordena por preço crescente
ordering	-preco	Ordena por preço decrescente (prefixo -)

Tabela 14.1 — Parâmetros de query string disponíveis no ProdutoViewSet

15. Testando com o REST Client (VS Code)

A extensão REST Client permite escrever e executar requisições HTTP diretamente de arquivos .http dentro do VS Code, sem precisar sair do editor.

15.1 Instalação

- Abra o VS Code;
- Acesse a aba Extensões (Ctrl+Shift+X);
- Pesquise "REST Client" e instale a extensão de Huachao Mao;
- Crie um arquivo com extensão .http (ex.: api_tests.http) na raiz do projeto.

15.2 Arquivo api_tests.http — Clientes

```
### Variáveis globais
@base = http://localhost:8000/amazon_api
@contentType = application/json

#####

### CLIENTES
#####

### 1. Listar todos os clientes
GET {{base}}/clientes/
Accept: application/json

###

### 2. Criar um novo cliente
POST {{base}}/clientes/
Content-Type: {{contentType}}

{
  "nome":      "Maria da Silva",
  "email":     "maria@email.com",
  "telefone": "(84) 91111-2222"
}

###

### 3. Buscar cliente por ID
GET {{base}}/clientes/1/
Accept: application/json

###

### 4. Atualizar parcialmente (PATCH)
PATCH {{base}}/clientes/1/
Content-Type: {{contentType}}

{
  "telefone": "(84) 99999-0000"
}

###
```

```
### 5. Remover cliente
DELETE {{base}}/clientes/1/
```

Arquivo 15.1 — Seção de Clientes em api_tests.http

15.3 Arquivo api_tests.http — Vendedores

```
#####
### VENDEDORES
#####

### 6. Listar todos os vendedores
GET {{base}}/vendedores/
Accept: application/json

###

### 7. Criar um vendedor
POST {{base}}/vendedores/
Content-Type: {{contentType}}

{
    "nome":      "Carlos Menezes",
    "email":     "carlos@loja.com",
    "cpf_cnpj":  "123.456.789-00",
    "telefone": "(84) 98765-4321",
    "avaliacao": "4.80",
    "ativo":     true
}

###

### 8. Filtrar vendedores ativos
GET {{base}}/vendedores/?ativo=true&ordering=-avaliacao
Accept: application/json

###

### 9. Busca textual
GET {{base}}/vendedores/?search=carlos
Accept: application/json

### 10. Atualizar avaliação de um vendedor
PATCH {{base}}/vendedores/1/
Content-Type: {{contentType}}

{
    "avaliacao": "3.50"
}
```

15.4 Arquivo api_tests.http — Produtos

```
#####  
### PRODUTOS  
#####  
  
### 11. Listar todos os produtos  
GET {{base}}/produtos/  
Accept: application/json  
  
###  
  
### 12. Criar um produto  
POST {{base}}/produtos/  
Content-Type: {{contentType}}  
  
{  
    "nome":      "Teclado Mecânico RGB",  
    "descricao": "Switches blue, iluminação RGB por tecla.",  
    "preco":     "349.90",  
    "estoque":   50,  
    "categoria": "eletronicos",  
    "disponivel": true  
}  
  
###  
  
### 13. Filtrar por categoria e ordenar por preço  
GET {{base}}/produtos/?categoria=eletronicos&ordering=preco  
Accept: application/json  
  
### 15. Atualizar estoque (PATCH)  
PATCH {{base}}/produtos/1/  
Content-Type: {{contentType}}  
  
{  
    "estoque": 45  
}  
  
###  
  
### 16. Substituição completa (PUT)  
PUT {{base}}/produtos/1/  
Content-Type: {{contentType}}  
  
{  
    "nome":      "Teclado Mecânico RGB Pro",  
    "descricao": "Versão Pro com switches red silenciosos.",
```

```
"preco":      "449.90",  
"estoque":    30,  
"categoria":  "eletronicos",  
"disponivel": true  
}
```

Arquivo 15.3 — Seção de Produtos em api_tests.http

💡 Para executar uma requisição no REST Client, clique no link 'Send Request' que aparece acima de cada bloco **###** no VS Code. A resposta é exibida em um painel lateral. Use **###** para separar requisições no mesmo arquivo.

15.5 Dicas de Produtividade com REST Client

Recurso	Como usar
Variáveis de ambiente	@base = http://... — define URL base reutilizável em todo arquivo
Múltiplas requisições	Separe cada requisição com ### no mesmo arquivo .http
Salvar resposta	Clique em Save Response Body para exportar o JSON retornado
Autenticação JWT	Adicione: Authorization: Bearer {{token}} antes do corpo
Histórico	Ctrl+Alt+H — exibe histórico de requisições enviadas

Tabela 15.1 — Recursos úteis da extensão REST Client no VS Code

16. Testando com Insomnia / Postman

Insomnia e Postman são clientes REST com interface gráfica dedicada, ideais para testar APIs de forma organizada. Ambos permitem criar coleções de requisições, definir variáveis de ambiente e exportar/importar configurações. A seguir, os exemplos são apresentados de forma genérica — aplicáveis a ambas as ferramentas.

16.1 Configurando o Ambiente

Antes de criar as requisições, defina um ambiente (Environment) para centralizar a URL base e evitar repetição:

Variável	Valor
base_url	http://localhost:8000/amazon_api
content_type	application/json

Tabela 16.1 — Variáveis de ambiente sugeridas

Com as variáveis definidas, use a sintaxe `{{base_url}}` nas URLs das requisições.

16.2 Coleção de Requisições — Vendedores

COLEÇÃO: Amazon API → PASTA: Vendedores

1. GET Listar Vendedores
URL: `{{base_url}}/vendedores/`
Params: (opcional) `ativo=true` | `ordering=-avaliacao`
2. POST Criar Vendedor
URL: `{{base_url}}/vendedores/`
Header: Content-Type: `application/json`
Body (JSON):
{
 "nome": "Ana Lima",
 "email": "ana@mercado.com",
 "cpf_cnpj": "987.654.321-00",
 "avaliacao": "4.95",
 "ativo": true
}
3. GET Buscar Vendedor por ID
URL: `{{base_url}}/vendedores/1/`
4. PATCH Atualizar Parcialmente
URL: `{{base_url}}/vendedores/1/`
Body: `{ "avaliacao": "4.20" }`

5. DELETE Remover Vendedor
URL: {{base_url}}/vendedores/1/

Arquivo 16.1 — Coleção Vendedores no Insomnia/Postman

16.3 Coleção de Requisições — Produtos

COLEÇÃO: Amazon API → PASTA: Produtos

1. GET Listar Produtos
URL: {{base_url}}/produtos/
Params: categoria=eletronicos | ordering=preco | search=teclado
2. POST Criar Produto
URL: {{base_url}}/produtos/
Header: Content-Type: application/json
Body (JSON):
{
 "nome": "Mouse Sem Fio",
 "descricao": "Mouse wireless 2.4GHz, DPI ajustável.",
 "preco": "129.90",
 "estoque": 100,
 "categoria": "eletronicos",
 "disponivel": true
}
3. GET Buscar Produto por ID
URL: {{base_url}}/produtos/1/
4. PATCH Atualizar Preço e Estoque
URL: {{base_url}}/produtos/1/
Body:
{
 "preco": "119.90",
 "estoque": 95
}
5. PUT Substituição Completa
URL: {{base_url}}/produtos/1/
Body: (todos os campos obrigatórios)
6. DELETE Remover Produto
URL: {{base_url}}/produtos/1/

Arquivo 16.2 — Coleção Produtos no Insomnia/Postman

16.4 Interpretando os Códigos de Status HTTP

Código	Status	Quando ocorre
200 OK	Sucesso	GET, PUT, PATCH bem-sucedidos
201 Created	Criado	POST bem-sucedido — recurso criado
204 No Content	Sem conteúdo	DELETE bem-sucedido
400 Bad Request	Requisição inválida	Dados inválidos ou ausentes no corpo
404 Not Found	Não encontrado	ID inexistente na URL
405 Method Not Allowed	Método inválido	Método HTTP não suportado pelo endpoint

Tabela 16.2 — Códigos de status HTTP comuns na API

16.5 Exportando e Compartilhando Coleções

Tanto o Insomnia quanto o Postman permitem exportar as coleções para um arquivo JSON e importá-las em outra máquina. Isso é útil para compartilhar os testes com outros membros da equipe.

Ação	Insomnia	Postman
Exportar	Preferences → Data → Export Data	File → Export → Collection
Importar	File → Import → From File	File → Import → Upload Files
Variáveis	Manage Environments → Add	Environments → Add
Histórico	Aba History (painel esquerdo)	Aba History (painel esquerdo)

Tabela 16.3 — Comparativo de funcionalidades entre Insomnia e Postman

17. Resumo da Extensão

Esta extensão de aula consolidou o fluxo completo para adicionar novos modelos a uma API Django REST Framework e testá-los com três ferramentas distintas:

Pas so	Componente	Responsabilidade
1	models.py (Vendedor)	Definir a tabela vendedores com campos de cadastro e avaliação
2	models.py (Produto)	Definir a tabela produtos com choices de categoria e campos de estoque
3	makemigrations / migrate	Criar as tabelas no banco de dados PostgreSQL
4	serializers.py	Converter objetos Vendedor e Produto em JSON (e vice-versa)
5	views.py (ViewSets)	Implementar CRUD completo com filtros, busca e ordenação
6	urls.py (Router)	Registrar as rotas /vendedores/ e /produtos/ automaticamente
7	Swagger UI	Documentar e testar endpoints via navegador sem código adicional
8	REST Client (VS Code)	Testar requisições HTTP dentro do editor com arquivos .http

Pas so	Componente	Responsabilidade
9	Insomnia / Postman	Organizar testes em coleções reutilizáveis com variáveis de ambiente

Tabela 17.1 — Resumo do fluxo de desenvolvimento da extensão